02
Basic Functional Programming
Values, functions, higher-ordering, types

Mirko Viroli
mirko.viroli@unibo.it

C.D.L. Magistrale in Ingegneria e Scienze Informatiche
Alma Mater Studiorum—Università di Bologna, Cesena

a.a. 2025/2026

Functional Programming in this course

Part 1

- brief overview of FP, basic "pure FP" in Scala
- function types, pattern matching, sum/product types

Part 2

- generics
- few simple data types and algorithms
- Lambda calculus theory
- confluence and evaluation order
- lazy evaluation in Scala, streams

Part 3

- module types
- type classes
- monads

Later

- integration with OOP: full Scala language
- relationships with LP: monadic view of Prolog resolution

Outline

1. Motivations
2. The Scala Programming Language: first round
3. Pure and basic functional programming in Scala 3

Is OO enough?

Limits of pure OO approaches

(la maggior parte dei linguaggi OOP ha troppo boilerplate)

- "everything is an object" can be verbose, and not "fitting"
- implementing functions with "1 method"-classes (so-called, functional interfaces) is verbose
- (note that verbosity implies inertia in usage)
- not declarative: focus on step-by-step behaviour, rather than on goal

Non tutte le astrazioni nel software rappresentano entità con uno stato e un comportamento (gli "oggetti" del mondo reale). Molti problemi sono modellati in modo molto più naturale in modo funzionale: da un input si ottiene un output, senza modificare alcuno stato interno.

Se un'astrazione richiede troppo boilerplate, la verbosità crea inerzia e le persone smettono di usarla. Fornire una sintassi snella e concisa (come le lambda) abbatte questa barriera e rende naturale l'uso continuo delle buone pratiche di programmazione.

What can "functional programming" do for us?

FP non viene inventato in risposta ai problemi di OOP: nasce alla fine degli anni '60

- a smooth abstraction for computing information
- nice techniques to abstract "internal execution" (i.e. imperative) aspects
- nice techniques to support concurrency orthogonally (see PCD)
- higher confidence in the correctness of code, by construction

$\Rightarrow$ possible disadvantage for software engineering:
functional code is terser and requires higher competence

Functional ideas outside functional languages

Ideas of functional languages, recently brought e.g. in OO languages

- "no side-effects", becoming "prefer immutability"
- "everything is a function", becoming "use lambdas, use factories to create objects"
- "higher-order functions", becoming "use the Strategy pattern to achieve generality"
- "abstract evaluation order", becoming "use lazy APIs" (as with streams)

"astrarre l'ordine di valutazione" indica la possibilità di disaccoppiare la definizione di un calcolo dal momento reale della sua esecuzione.

la computazione parte con qualsiasi operazione terminale(collect, reduce, etc)

Lambdas and Streams in Java 8+: FP-like programming

```java
public static void main(String[] args) {
    // using a 'lambda function' to express a functional strategy
    final Comparator<Integer> comparator = (a, b) -> a - b;
    final Set<Integer> set = new TreeSet<>(comparator); (strategy pattern)
    set.addAll(Arrays.asList(10, 20, 30, 40, 5, 6, 7, 60, 70));
    System.out.println(set);

    // declaratively working with streams: note str is an 'infinite' sequence!
    final Random random = new Random();
    final Stream<Integer> str = Stream.generate(() -> random.nextInt(6))
                                      .map(i -> i + 1);
    System.out.println(str.limit(20).collect(toList()));

    // simulating large-scale data processing with parallel streams
    final List<Double> l = IntStream.range(0, 10000000)
            .mapToObj(x -> Math.random())
            .collect(toList()); In questo caso la computazione parte con collect(toList()), che forza lo
    // sequential                stream a generare i 10 milioni di elementi casuali e a salvarli nella lista l.
    long time = System.currentTimeMillis();
    System.out.println(l.stream().reduce((a,b)->Math.max(a,b)));
    time = System.currentTimeMillis() - time;
    System.out.println("Time: " + time);
    // parallel
    long time2 = System.currentTimeMillis();
    System.out.println(l.stream().parallel().reduce((a,b)->Math.max(a,b)));
    time2 = System.currentTimeMillis() - time2;
    System.out.println("Time2: " + time2);
    System.out.println("Gain: " + (((double)time)/time2));
}
```

Main features of lambdas and streams

Java Lambdas

- functional interfaces are already popular: see Strategy, Iterator, Template Method
- with lambdas you have a smooth syntax to quickly implement them
- it becomes natural to use them pervasively, as in FP

Java Streams

- enable a fully-declarative approach to manipulate collections
- side-effects are neglected
- allow the programmer to transparently take advantage of multi-core architectures
- apparently enrich the "language": can the approach be applied elsewhere?

$\Rightarrow$ a disadvantage: `Stream` API very different from `List`'s, why?

$\Rightarrow$ aren't `Stream` and `List` two ways of dealing with sequences?

Outline

1 Motivations

2 The Scala Programming Language: first round

3 Pure and basic functional programming in Scala 3

Scala: introduction

il linguaggio è stato progettato per adattarsi in modo smooth all'aumentare della complessità del software. In informatica, la "scalabilità" di un linguaggio non si riferisce solo alle prestazioni di rete o di calcolo, ma alla capacità del linguaggio di gestire sia piccoli script che grandi architetture software

Briefly: more than a modern Java

- strongly-typed PL that compiles to Java bytecode (+ native/JS)
- "**Sca**lable **La**nguage", i.e., designed ~~to smoothly address~~ per gestire con facilità increasing complexity

Some history

- design of Scala started in 2001 at EPFL by Martin Oderski;
- the goal was to design a PL better than Java [a] that was both object-oriented and functional;
- first public release in 2004. Version 2 in 2006. Version 3 in 2021 – we will use it (3.3.5)!
- it is a top popular JVM-language, with Java and Kotlin
- Java 8+, C#, Kotlin: all inherited their FP-nature from Scala
- Scala is the pioneer of OOP+FP, it keeps experimenting new mechanisms, it takes from other languages

[a] `http://www.artima.com/scalazine/articles/origins_of_scala.html`

Scala: distinguishing features

Key characteristics

- smooth integration of OOP and FP
  - ultimately, OOP-inside (due to JVM), with FP sugar on top
- strong, expressive type system
  - structural types, intersection types, sum types, self-types, ...
  - type inference is pervasive
- purely object-oriented (class-based)
  - every value is an object, every operation is a method call
  - single class inheritance, (multiple) mixin-composition via `traits`
- syntactic sugar and syntactic flexibility, enabling expressive DSLs (Domain-Specific Language)
- designed to express common programming patterns in a concise and typesafe way
- support for high-level FP patterns, like monadic constructions and higher-order kinds

La mixin-composition consente a una classe di ereditare da una classe base e contemporaneamente incorporare (mix in) molteplici trait.

Scala: trends/perspectives

Scala in academia

- Scala is one of the mainstream languages with most research so far
- Scala is the most sophisticated porting of FP (onto the JVM)
- Differently from Java, Scala targets a smaller niche

Scala in industry

- Scala is used by several companies, in Italy as well

$\Rightarrow$ some hired our students too

- Scala programmers are somewhat an elite (top paid in 2020!)[a]

[a] https://www.businessinsider.com/highest-paying-programming-languages-stack-overflow-developer-survey-2020-5

Scala and distributed system development

- Frameworks such as Akka (actors), Spark (big-data/cluster-computing), Kafka (stream processing) are written in Scala

Scala as a teaching tool

- Learn Scala and you will have a profound understanding of modern PL and FP
- Learn Scala and will be ready to quickly learn Kotlin/Closure/Haskell if needed

Tooling

SBT: Scala Build Tool

- well integrated with Scala, Java, and test suites: a competitor of Gradle

$\Rightarrow$ we will use it, mostly

REPL: Read-evaluate-print loop

- a command-line tool, usable also from IDEs
- it evaluates expressions, dynamically growing a scope of variables

$\Rightarrow$ we shall use it this week

IntelliJ IDEA: our reference IDEs

- VSCode a possible alternative

Execution platform

- Scala-to-JVM: by default, the natural option
- Scala-to-JavaScript: very good support (see e.g., Scastie web playground)
- Scala-to-native: also viable and working, though with significant memory footprint

Outline

1 Motivations

2 The Scala Programming Language: first round

3 Pure and basic functional programming in Scala 3

FP concepts to start with

Key ideas

- computation is expressed by a set of functions, possibly split in modules
- functions ~~perform pure~~ (hanno un comportamento di tipo) "many-input/one-output" ~~behaviour~~
- no side-effects (dealing with I/O over console is considered a hack)
- ~~values include~~ (I valori includono:) standard primitive ones, functions, records (Product Types), sums (Sum Types)/unions
- no imperative constructs, recursion as the single "iteration" mechanism

Start with primitive types (and a pseudo-grammar)

A differenza di uno statement (istruzione), che si limita a compiere un'operazione o a modificare uno stato senza ritornare un risultato, un'espressione calcola ed emette sempre un valore di ritorno.

In Scala, quasi tutto è un'espressione, ma gli statement esistono sotto forma di espressioni il cui scopo principale è produrre un side-effect e che restituiscono il tipo Unit.

$T ::= \texttt{Int} \mid \texttt{Double} \mid \texttt{Boolean} \mid \texttt{String} \mid \ldots, \quad e ::= v \mid v_1\ op\ v_2 \mid \ldots$

```
1  // values can be associated to names
2  // namely, non-modifiable variables
3  // (of course, there's no side effect)
4  val v = 1
5  // types indication is optional (typically inferred)
6  val w: Int = 1
7  // right-hand side contains a whole expression
8  val z = v + 5
9  // usual procedure to print on console (a non-FP hack)
10 println(z)
```

Start with primitive types: details

```scala
val a = 1
println("result is " + a) // Java-style concatenation
println(s"result is $a") // string interpolation, with variable evaluation
println(s"result is ${a+1}") // string interpolation, with expr. eval.

// primitive types as expected over the JVM
val i: Int = 10 + 5 // as in Java, to be read +(10,5)
val l: Long = 100_000_000_000L // as in Java
val d: Double = 5.4 * 1.3 // as in Java
val f: Float = 3.0f // as in Java
val b: Boolean = true && false // as in Java
val s: String = "hello" concat " to all" // String methods as operators
val n: String = null // null can be passed to "objects", sorry
```

Primitive types

In Scala ogni valore è un oggetto ed ogni operazione è una chiamata a un metodo.

Notes

- Scala has richer type system than Java, but for now we stick to `Int`, `Boolean`, `String`, `Unit`
- Scala has no true difference between primitive and non-primitive types
- Scala has stronger type inference: in variable/values, type declaration are typically omitted
- `println` is not really a function, but we shortly need it just to log to console

On style: where to put spaces?

1. never in between function name and parameters list
2. always before and after an operator
3. never use them after a "(" or before ")"
4. always after a ",", but not before it
5. always in between different syntactic elements

$\Rightarrow$ or simply, carefully mimic slides...

Our first tool, the REPL

Scala Read-Eval-Print-Loop

- a console-oriented tool to evaluate even complex Scala expressions
- very powerful, typically integrated in IDEs
- very useful also (not only) at the beginning
- it inspired `JShell`

Mini tutorial for now

- type `scala` (or better `sbt console`) from the command-line, check correct version
- type your expression and see type and result
- commands: `:quit`, `:reset`, `:type`

A note on REPL vs IDE

- in an IDE should add preamble "`object M extends App:`"...
- to generate an executable `M.class`, or to work on an IDE

Function types, and anonymous functions (a.k.a "function literals")

$T ::= (T_1, \ldots, T_n) \Rightarrow T_0 \mid \ldots$

$e ::= (x_1, \ldots, x_n) \Rightarrow e_1 \ldots e_n \mid e_f(e_1, \ldots, e_n) \mid \ldots$

```
1  // Defining and assigning an Int
2  val i: Int = 5
3  // Definition and assigning a function:
4  // terminology: define a function variable
5  // type: (Int, Int) => Int,
6  // assigned value: an anonymous function (function literal)
7  val f: (Int, Int) => Int = (x: Int, y: Int) => x + y
8  // standard syntax for function call with "value passing"
9  val res = f(5, 10)
10 println(res)
```

Function types, and anonymous functions (a.k.a "function literals")

$T ::= (T_1, \ldots, T_n) \Rightarrow T_0 \mid \ldots$

$e ::= (x_1, \ldots, x_n) \Rightarrow e_1 \ldots e_n \mid e_f(e_1, \ldots, e_n) \mid \ldots$

```
1  // various syntax for function literals
2  val f1: (Int, Int) => Int = (x: Int, y: Int) => x + y // typed with typed val
3  val f2 = (x: Int, y: Int) => x + y // typed with untyped val
4  val f3: (Int, Int) => Int = (x, y) => x + y // untyped, with typed val
5  val f4: (Int, Int) => Int = _ + _ // expression with placeholders, with typed val
6  val f5: (Int, Int) => Int = f4  // a function variable can be used as value
7  // val f6 = _ + _ // won't type-check
8
9  println(s"${f1(10, 5)}, ${f4(10, 5)}") // (15,15)
10
11 val g: Int => Int =
12   (x: Int) =>  // a function literal with an imperative body
13     println("hello") // a statement (with some side-effect)
14     x + 1 // last statement is the return
15
16 println(g(10)) // gives 11, but also prints "hello"
```

Functions: function types vs function values

Le funzioni sono da considerare come un nuovo tipo di dato, oltre ai primitivi

Function type

- written $(T_1, \ldots, T_n) \Rightarrow T_0$ (special cases $T_i \Rightarrow T_o$ and $() \Rightarrow T_o$)
- is the type of functions (from n inputs to 1 output)

Function value (also called function literal, in Java it is a lambda)

- written $(x_1, \ldots, x_n) \Rightarrow e$ (special cases $x \Rightarrow e$ and $() \Rightarrow e$)
- denotes an actual function to be called

. . . problems/opportunities come from recursive use

il function literal definisce la funzione senza darle un nome, a prescindere che venga assegnata o meno a una variabile. L'assegnamento a una variabile è soltanto uno dei modi in cui un function literal può essere utilizzato (l'altro modo è passarlo senza assegnamento).

Functions

Notes

- function values are called *function literal* in Scala (lambdas in Java)
- do not make confusion with syntax: in `f1` to `f4` the part before = is val declaration, the part after is the function itself
- there are four styles for expressing a function (value):
  1. by a `val`, i.e., by a value containing it (see `f5`)
  2. by a typed function literal, i.e., explicitly typing its arguments (see `f1`, `f2`)
  3. by an untyped function literal, i.e., not typing its arguments (see `f3`)
  4. by an expression with underscores, i.e., where underscores are interpreted as successive arguments of a function literal (see `f4`)
- 0-arg list is denoted `()=>..`, 1-arg `(x)=>..` or `x=>..` or `_=>..`
- the body is typically a single expression
- sometimes we make it imperative, by placing many expressions in multiple lines... the last expression is a return (could use curly brackets)

Higher-order functions

$T ::= (T_1, \ldots, T_n) \Rightarrow T_0 \mid \ldots$

$e ::= (x_1, \ldots, x_n) \Rightarrow e_1 \ldots e_n \mid e_f(e_1, \ldots, e_n) \mid \ldots$

```
1  // non-higher-order, i.e., "first order"
2  val f = (x: Int, y: Int) => x + y
3  // higher-order function: some input/ouput is a function itself
4  val h: (Int, Int, (Int, Int) => Int) => Int = (a, b, f) => f(a, b)
5
6  println(h(10, 5, _ * _)) // 50
7  println(h(10, 5, (a, b) => a + b )) // 15
8  println(h(10, 5, f)) // 15
9
10 val i: (Int, Int => Int, Int => Int) => Int =
11   (n, f1, f2) => f1(f2(n))
12
13 println(i(10, _ + 1, _ * 2)) // 21
14
15 val l: (Int => Int) => (Int => Int) =
16   f => (i => f(f(i))) // without parens: f => i => f(f(i))
17
18 println(l(_ + 1)(10)) // 12, see currying next..
```

Higher-order functions

Notes

Sono funzioni che prendono in input altre funzioni (per passare una funzione come parametro ad un'altra, bisogna definire il tipo della funzione nella dichiarazione della funzione che la riceve in input)

- *higher-order* functions: functions having a function as argument or return type
- a quintessential "pattern" of FP
- it is a natural way to make a function more reusable, more configurable
- essentially, the FP-version of Strategy pattern (or rather, vice-versa)
- when the return type is a function, we have the currying mechanism
- recall that `=>` is right-associative, hence:
  "`a => b => c`" is the same as "`a => (b => c)`"

Match and cases, and use as 1-arg function values

$e ::= \{\texttt{case}\ p_1\ \texttt{=>}\ e_1\texttt{;}\ \ldots\} \mid e\ \texttt{match}\ \{\ \texttt{case}\ \ldots\} \mid \ldots$

```
1  // matching against patterns, i.e., a switch expression
2  val v = 5
3  val res = v match
4    case n if n > 0 => "pos"
5    case 0 => "zero"
6    case _ => "neg" // default case, makes it a total function
7  println(res) // "pos"
8
9  // case-based function (a partial function in this case)
10 // val f = (i: Int) => i match // alternative header
11 val f: Int => String = _ match
12   case n if n > 0 => "pos"
13   case 0 => "zero"
14
15 println(f(1)) // pos
16 println(f(0)) // zero
17 // f(-1) raises a scala.MatchError
18
19 val g: Int => String = { case n if n > 0 => "pos" } // 1-liner with {}
20 println(g(1)) // pos
```

si tratta di una classe di eccezione runtime che viene sollevata in Scala quando un meccanismo di pattern matching fallisce perché il valore valutato non corrisponde a nessuno dei casi (case) specificati e non è stato definito un caso di default

Match and cases

Se definisci una funzione basata su case senza coprire tutti i possibili casi, crei una funzione parziale

la funzione prende un solo parametro in ingresso.

Notes

(indica l'insieme di case che definiscono i vari pattern da verificare)

- a `case` block (`{case` p_1 `=>` e_1`; ...}`) defines a partial unary function, which is a particular case of function that can fail on some input, raising an exception (`scala.MatchError`) – but we typically won't throw / catch it
- cases have a very flexible syntax (will see later), now we consider: values, variables, variables with `if` condition
- `match` in front of a `case` block is used to apply such partial function intuitively
- "`_ match`" and cases with significant indentation are the more idiomatic style for Scala 3
  - use curly brackets if need a more concise, e.g., single-line, syntax
- if the left-hand side of a match is a variable, it is treated as a fresh one
- cases are processed from top to bottom, only the first that matches gets evaluated

quando usi una variabile (scritta con l'iniziale minuscola) sul lato sinistro di un case in un match, Scala non controlla se esiste già una variabile con quel nome, ma ne crea una completamente nuova (fresh) che "cattura" e sovrascrive temporaneamente qualsiasi valore precedente.

On indentation styles in Scala

```scala
object Indentation:

  // idiomatic Scala 3 significant indentation
  val res = 5 match
    case n if n > 0 => "pos"
    case 0 => "zero"

  // indentation with end marker (useful in complicated cases)
  val res2 = 5 match
    case n if n > 0 => "pos"
    case 0 => "zero"
  end res2

  // Scala 2  (and Java) style (we avoid it unless necessary)
  val res3 = 5 match {
    case n if n > 0 => "pos"
    case 0 => "zero"
  }

  // Single line curly braces with semi-colon
  val res4 = 5 match { case n if n > 0 => "pos"; case 0 => "zero" }

  // Single parameter of a function could be indented with colon
  println:
     5 + 10

end Indentation // showcasing end marker for object
```

On indentation styles in Scala

Notes

- a very debated transition from Scala 2 to Scala 3
- ultimately, it works very ok
- our stance: definitely use it!

Guidelines for this course: use significant indentation

- use curly braces only for 1-liners only where needed
- try not to complicate, hence use significant indentation
- only where complicated, use end markers (will see in the future)
- use indented argument only with imperative-like code

Defined functions, i.e., methods

$D ::= \texttt{def}\ f(x_1 : T_1, \ldots, x_n : T_n) : T_0 = e \mid \ldots$

```scala
// defined function, essentially using UML-like syntax
def square(d: Double): Double = d * d

println(square(3.0)) // 9.0

// mostly equivalent code with a val
val square2: Double => Double = d => d * d
println(square2(3.0)) // 9.0

// defined function, using pattern matching, with recursion
def factorial(n: Int): Int = n match
  case 0 | 1 => 1 // 0 or 1 gets mapped to 1
  case _ => n * factorial(n - 1) // otherwise

println(factorial(6)) // 720
```

Defined functions, i.e., methods

Notes

- a method is not very different from a `val` holding a function: it will allow additional features (currying, generics, annotations, nesting), and it will be translated into a bytecode method, and will be part of classes/objects
- methods necessarily need to define types of inputs, return type is optional (but typically provided)
- methods can nest in each other (not possible in Java)
- methods can call themselves recursively, but recall this is optimised to be efficient only if the recursion is "tail"
  - a recursion is *tail* if all the recursive calls, when executed, are the last expression evaluated before returning
  - if you are not sure whether your call is tail, use the annotation: it makes code not compile otherwise

Defined functions: nesting

$D ::= \texttt{def}\ f(x_1 : T_1, \ldots, x_n : T_n) : T_0 = e \mid \ldots$

```
1  // nested function, note indentation
2  def abs(n: Int): Int =
3    def sign(m: Int): Int = m match
4      case i if i > 0 => 1
5      case 0 => 0
6      case _ => -1
7    n * sign(n)
8
9  println(s"${abs(-2)}, ${abs(2)}, ${abs(0)}") // 2,2,0
10
11 // tail recursion check, as a nested function
12 def factorial2(n: Int): Int =
13   @annotation.tailrec // checks only if optimisation is possible
14   def _fact(n: Int, acc: Int): Int = n match
15     case 0 | 1 => acc
16     case _ => _fact(n - 1, n * acc)
17   _fact(n, 1)
18
19 println(factorial2(6)) // 720
```

Currying

$$D ::= \texttt{def}\ \ f(x_1 : T_1, \ldots, x_n : T_n)(\ldots)(\ldots) : T_0 \ = \ e \mid \ldots$$

```
1  // standard function with no currying
2  def mult(x: Double, y: Double): Double = x * y
3
4  // function with currying
5  // curriedMult has actually type: Double => (Double => Double)
6  def curriedMult(x: Double)(y: Double): Double = x * y
7
8  // slightly different syntax at the call side..
9  println(mult(10, 2)) // 20
10 println(curriedMult(10)(2)) // 20
11
12 // curriedMult can be partially applied!
13 val twice: Double => Double = curriedMult(2)
14
15 println(twice(10)) // 20
16
17 // => is right-associative, hence it is equal to:
18 //val curr...: Double => (Double => Double) = x => (y => x*y)
19 val curriedMultAsFunction: Double => Double => Double = x => y => x * y
20
21 println(curriedMultAsFunction(10)(2)) // 20
```

Scala ti permette di usare una sintassi compatta. Il tipo "Double" in coda specifica il risultato dell'espressione finale "x * y". Tuttavia, il compilatore gestisce questo metodo a due livelli:
- Se fornisci entrambi gli argomenti, il metodo viene valutato del tutto e restituisce un Double.
- Se fornisci solo la prima lista di argomenti, il metodo si arresta al primo passaggio e restituisce una nuova funzione di tipo "Double => Double" (applicazione parziale).

In questa espressione, la valutazione avviene in due fasi successive (a tappe) grazie alla natura sequenziale delle funzioni curried ed all'associatività a destra dell'operatore "=>". Come si scompone la struttura:
> Sia il tipo che la definizione si associano a destra: Double => Double => Double equivale a Double => (Double => Double).
> Implementazione: x => y => x * y equivale a x => (y => x * y). Si tratta quindi di una funzione esterna che accetta x e restituisce una funzione interna che accetta y.

Currying

def curriedMult(x: Double)(y: Double): Double = x * y
- Perché il tipo di ritorno finale è Double? Perché un metodo definisce l'operazione completa. Il tipo di ritorno descrive cosa restituisce il metodo quando la sua esecuzione è giunta al termine (cioè il risultato di "x * y").
- Come fa a restituire "Double => Double" senza scriverlo? Quando scrivi curriedMult(2) passando solo il primo gruppo di argomenti, entra in gioco il compilatore con un meccanismo di zucchero sintattico chiamato Eta-Expansion. Il compilatore vede che la seconda lista di parametri (y: Double) non è stata fornita e trasforma automaticamente la chiamata in un oggetto funzione al volo (y => curriedMult(2)(y)), il cui tipo è Double => Double.

In Scala, il currying è una tecnica che trasforma una funzione che accetta più argomenti insieme in una serie di funzioni che accettano un singolo argomento ciascuna: invece di applicare tutti gli argomenti in una volta sola, la funzione viene valutata un pezzo alla volta, restituendo una nuova funzione a ogni passaggio fino a ottenere il risultato finale.

Notes

- *Currying* is the technique ~~by which~~ (mediante la quale si eseguono) multiple application to arguments ~~are performed~~ ~~by~~ (tramite) successive evaluation of a function and its results
- it is basically a shorthand for a function returning a function returning a function...
- when a function is curried, you can have partial application, which returns a function
- sometimes, currying is used when it is convenient to organise arguments into sub-groups, for ease of reading or for other technical reasons (we will see why)

val curriedMultAsFunction: Double => (Double => Double) = x => y => x * y
- Perché la firma esplicita è Double => (Double => Double)? Perché qui non c'è un metodo con liste di parametri multiple separate, ma una struttura dati ad oggetti annidata: curriedMultAsFunction è un primo oggetto funzione che accetta un Double. Quando lo invochi con (2), l'oggetto esegue il suo codice e restituisce un secondo oggetto funzione.
La firma descrive esattamente il tipo di questo oggetto: "una funzione che prende un Double e restituisce una funzione che prende un Double e restituisce un Double".

Algebraic data types

ATTENZIONE: Sum e Product types non sono da vedere come concetto di ereditarietà (Product types specializzano il Sum type) oppure interfaccia (sum types)

Flexibly constructing types

- need a way to express data structures (from simple, to complex)
- An algebraic data is a composite type...
- ...defined by composition, possibly at multiple stages, from atomic types

Permettono di definire strutture dati complesse combinando due concetti fondamentali: Product Types (Tipi Prodotto) e Sum Types (Tipi Somma).

Two main cases of composition

Due modi per modellare un tipo di dato algebrico

1. Definito in Scala con "case class"
   - Product types (set-based cartesian product), aka, records
   - $\Rightarrow$ a value contains a part of type T_1 and one of part T_2

Un Product Type rappresenta una combinazione di valori: un'istanza contiene contemporaneamente un dato di tipo T1 e un dato di tipo T2. Il nome deriva dal "prodotto cartesiano" poiché il numero totale di combinazioni possibili è il prodotto delle possibilità dei singoli tipi.

2. Definito in Scala con "enum"
   - Sum types (set-based ~~disjoint union~~ unione disgiunta), aka, cases
   - $\Rightarrow$ a value is either of type T_1 or T_2 (not from both, tough!)

Un Sum Type rappresenta una scelta esclusiva: un valore appartiene al tipo T1 oppure al tipo T2, ma non a entrambi. In matematica corrisponde all'unione disgiunta. Il numero di elementi totali è la somma degli elementi dei singoli tipi.

- using the two altogether makes quite a huge expressiveness...

Product types (aka, records)

$D ::= \texttt{case class}\ C(x_1 : T_1, \ldots, x_n : T_n) \mid \ldots, \quad T ::= C \mid \ldots, \quad e ::= C(e_1, \ldots, e_n) \mid \ldots$

```
1  // Introducing a new product type, aka, a record type
2  case class Point2D(x: Double, y: Double)
3  // Defining and assigning a variable of type Point2D
4  val p: Point2D = Point2D(10, 20)
5  println(p) // Point2D(10.0,20.0), string representation
```

Product types (aka, records): details

```scala
case class Point2D(x: Double, y: Double)
// Defining and assigning a variable of type Point2D
val p: Point2D = Point2D(10, 20)
println(p) // Point2D(10.0,20.0), string representation

// using matching to declaratively manipulate a Point
def rotate(p: Point2D): Point2D = p match
  case Point2D(x, y) => Point2D(y, -x)

// matching over couples
def sum(p1: Point2D, p2: Point2D): Point2D = (p1, p2) match
  case (Point2D(x1, y1), Point2D(x2, y2)) => Point2D(x1 + x2, y1 + y2)

// partial matching
def getX(p: Point2D): Double = p match
  case Point2D(x, _) => x
                  uso 'underscore' nel case del match quando un parametro non mi interessa
// a fluent setter that creates a new record...
def chgX(p: Point2D, x: Double): Point2D = p match
  case Point2D(_, y) => Point2D(x, y)

println(rotate(Point2D(10, 20))) // Point2D(20.0,-10.0)
println(sum(Point2D(10, 20), Point2D(3, 4))) // Point2D(13.0,24.0)
println(getX(Point2D(10, 20))) // 10
println(chgX(Point2D(10, 20), 11)) // Point2D(11,20)
```

Product types

Notes

- essentially, a record
- most importantly for Scala, can match against a record (using values, variables or underscores for its fields)
- the tuple notation `(p1,p2)` is actually equivalent to a case class `Tuple2(p1,p2)`
- in fact, matching works very well with records of records of records...

A glimpse of OOP vs FP

- note that we actually introduced almost all that is needed (data and functions) to handle somewhat an object
- though in FP we do not encapsulate data and functions but often keep them separated – in FP, that is a feature...

Sum types (with Scala 3 enums): full algebraic data types

$D ::=$ `enum` E`:` `...case` C`(..)`

```scala
// introducing a new sum type, defined by enumerating various cases
enum Person:
  case Student(name: String, year: Int)
  case Teacher(name: String, course: String)

// a value of type Person is either a Student OR a Teacher
val p: Person = Person.Student("mario", 2015)

// an algorithm/manipulation over Person
def name(p: Person): String = p match
  case Person.Student(n, _) => n
  case Person.Teacher(n, _) => n

println(name(p))
```

Full algebraic data types: details

```scala
// other examples

enum WeekDay: // a Java-like enumeration
  case Monday, Tuesday, Wednesday, Thursday, Friday

def preferredDay(d: WeekDay): Boolean =
  d == WeekDay.Friday // could have used matching instead

// A LinkedList of Int
enum IntList: // a recursive type
  case Cons(head: Int, tail: IntList)
  case Nil

def sum(l: IntList): Int = l match
  case IntList.Cons(h, t) => h + sum(t)
  case _ => 0

println(sum(IntList.Cons(10, IntList.Cons(20, IntList.Nil))))
```

Algebraic data types with Scala enums

Notes

- An "enum" defines a new type that is the "sum" of a number of product types
- Note that one can have a product type with 0 arguments, which is essentially a singleton
- Various types of Algebraic Data Types
  - An enumeration of singletons (which would be a sort of Java enumeration)
  - A sum of many records (which would be a hierarchy)
  - A sum with a recursive pattern (used to model structures like linked lists)
- Cases of an `enum` need to be qualified by the `enum` name
- Note that "Scala enums" cannot be applied to model any Algebraic Data Type, but it is enough for now – will see more powerful constructs later

Subtyping, i.e., inclusive polymorphism

La relazione di subtyping è:
- Riflessiva: ogni tipo è sottotipo di se stesso ('T <: T').
- Transitiva: se 'A <: B' e 'B <: C', allora 'A <: C'.

General notion

- the subtype relation `T <: R` is generally introduced in typed languages
- it is reflexive and transitive: i.e., a preorder
- it means that any value of type `T` can be passed where a value of `R` is expected

Subtyping by sum types

- a sum type is supertype of its elements
  - for example, `Student` is a subtype of `Person`
- matching is used at the receiving-site to intercept the subtype

Subtyping of functions

Quando è possibile dire che una funzione F1 è sottotipo di un'altra funzione F2?

- $(T_1, \ldots, T_n) \Rightarrow T_0$ <: $(R_1, \ldots, R_n) \Rightarrow R_0$ iff:
  - covariance on return: T_0 <: R_0 ↔ La funzione sottotipo deve restituire un valore di tipo uguale o sottotipo di quello richiesto.
  - contravariance on arguments: R_i <: T_i for each i ↔ La funzione sottotipo deve accettare in ingresso un argomento di tipo uguale o sopratipo rispetto a quello che le verrà passato.
- `val f: Student => Person = (p: Person) => Student("m", 2015)` ↔ Viene assegnata una funzione sottotipo (valida perchè ritorna un sottotipo ed accetta sopratipi)
- no matching is needed at the receiving-site

Summary of core part of FP-Scala: constructs

Types

$T ::=$	`Int` | `Double` | `Boolean` | `String` |	primitives
	`(`T_1`,...,`T_n`)=>`T_0 |	functions
	C | E	case classes, enums

Declarations

$D ::=$	`val` x`:`T `=` v |	value
	`def` f`(`x_1`:`T_1`,...,`x_n`:`T_n`):`T_0 `=` e |	function
	`case class` C`(`x_1`:`T_1`,...,`x_n`:`T_n`)` |	product type
	`enum` E`:` `...case` C`(..)`	algebraic data type

Expressions

$e ::=$	v | v_1 op v_2 | $e_f(e_1,\ldots,e_n)$ | $(x_1,\ldots,x_n)$`=>`$e_1 \ldots e_n$ |	functional
	`{case` p_1 `=>` e_1`; ...}` | e `match { case ...}` |	matching
	$C(e_1,\ldots,e_n)$	record

Summary of core part of FP-Scala: expressiveness

Types

- product types (cartesian product over sets) and algebraic data types (by sum) altogether allow modelling articulate data structures and application domains
- first-order functions allow to greatly generalise existing code into reusable components (see generics next)

Expressions and declarations

- very few constructs, sticking to a declarative (match-based) programming style
- additional constructs (`if`, blocks) can be used
- further syntactic sugar to handle evaluation order can be used

Modules and imports

Modules in FP

- a mechanism to group definitions under a module ("namespace")
- some may be public (to other modules) other private
- public definitions accessible via *module.defname* syntax
- Modules + Types are used to structure data types and algorithms

⇒ operators are inserted into a module with same name of the type

In Scala

- `object ModuleName:`
- can avoid syntax "`ModuleName.`" by an import: `import ModuleName.*`

In Scala REPL

- can use imports, can safely use `ctrl-v` to paste whole modules

Modules and imports

```scala
// An ADT: type + module
enum Person:
  case Student(name: String, year: Int)
  case Teacher(name: String, course: String)

object Person:
  def name(p: Person): String = p match
    case Student(n, _) => n
    case Teacher(n, _) => n

println(Person.name(Person.Student("mario", 2015)))

import Person.*

println(name(Student("mario", 2015)))

// a method outside the Person module
def isStudent(p: Person): Boolean = p match
  case Student(_, _) => true
  case _ => false

println(isStudent(Student("mario", 2015)))
```

TDD for Scala?

```scala
import org.junit.*
import org.junit.Assert.*

class RecordTest:

  case class Item(desc: String, code: Int)
  val record = Item("a", 10)

  @Test def testMatches() =
    assertTrue(record match {case Item(x, 10) => true; case _ => false})
    assertTrue: // significant indentation style
      record match
        case Item("a", 10) => true
        case _ => false

  @Test def testEquality() =
    assertEquals(record, Item("a", 10))
    assertNotEquals(record, Item("a", 11))

  @Test def testChangeCode() =
    def changeCode(item: Item)(f: Int => Int): Item = item match
      case Item(desc, code) => Item(desc, f(code))
    assertEquals(Item("a", 11), changeCode(Item("a", 10))(_ + 1))
```

TDD for Scala

Notes

- we will see more proper test frameworks for Scala
- JUnit can be used smoothly if you want to retain TDD
  - note we do not know what is a “class” in Scala
- in FP, a test function typically only has asserts
- could use the following SBT file (it activates JUnit plugin in VSCode)

```
scalaVersion := "3.3.5"

lazy val root = (project in file("."))
  .settings(
    name := "pps-code-scala",
    libraryDependencies += "com.github.sbt" % "junit-interface" % "0.13.2" % Test
  )
```

Wrap-up

FP so far

- can define some data types, i.e., simple to complex data structures
- computational behaviour expressed by functions and pattern-matching
- not sure how it "scales with complexity", but in fact it does

FP in next lessons

- will discuss the key property of declarativity
- will go to `Optional`/`List`/`Stream` data types